

Mean-Shift tracking

Marko Medved

I. INTRODUCTION

In this assignment, we implemented two different Bayesian trackers: the Kalman filter tracker and the particle filter tracker. We derived the input matrices for the Random Walk, Nearly Constant Velocity, and Nearly Constant Acceleration motion models. The Kalman filter tracker was tested on three artificial trajectories, while the particle filter tracker was evaluated on the VOT2014 dataset. For the particle filter tracker, we compared the performance of different motion models and determined the optimal dynamic noise parameter for all three models. Additionally, we tested the tracker with varying numbers of particles and explored its performance across different color spaces.

II. EXPERIMENTS

A. Kalman filtering

In this section, we derive the input matrices (see Appendix) for the Random Walk (RW), Nearly Constant Velocity (NCV), and Nearly Constant Acceleration (NCA) motion models. These models were tested on three different trajectories to evaluate their performance.

Figures 1, 2, and 3 show the trajectories and their filtered counterparts for the different motion models, as well as the effects of varying the dynamics noise constant (q) and the measurement covariance constant (r). In all three plots, we observe that when q is large and r is small, the filtered measurements closely follow the actual measurements. This is expected because a larger q assigns more uncertainty to the dynamics, leading the tracker to rely more heavily on the measurements for prediction.

On the other hand, when r is set high and q is set low, the tracker relies more on the motion model. This is evident because, in this case, the random walk struggles to follow the path quickly enough. For the NCV and NCA models, we observe that the paths start to deviate significantly from the measurements, particularly in the second plot. However, this is in general not a negative outcome. In scenarios where measurements are unreliable or sparse, the tracker can perform better by depending more on the motion model, which compensates for the poor measurements.

The optimal parameters depend on both the choice of motion model and the specific application. For the Random Walk model, it is advisable to rely more on the measurements, particularly when the movements are erratic. In such cases, values like $q = 5$ and $r = 1$ would be suitable. In contrast, for the Nearly Constant Velocity and Nearly Constant Acceleration models, where the movement is more predictable or dynamic, we can place more trust in the motion model, which means selecting a larger value for r .

B. Results of the basic particle filter tracker implementation

In this section, we implemented the particle filter tracker using the Nearly Constant Velocity motion model and color histograms for the visual model. The results for the entire VOT 2014 dataset are presented in Table I. For the implementation, we set the update speed to $\alpha = 0.0001$, the kernel size to $\sigma = 1$, and used 16 bins in the histogram. The dynamic noise constant

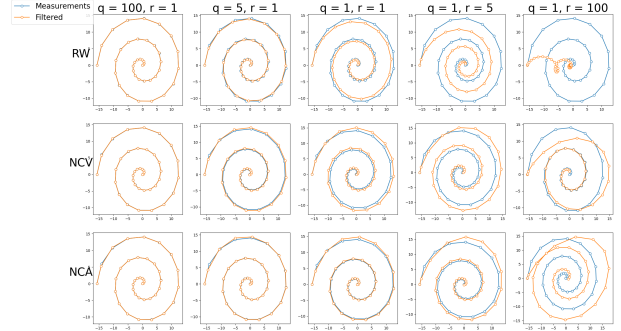


Figure 1. Kalman filtering method on the spiral curve

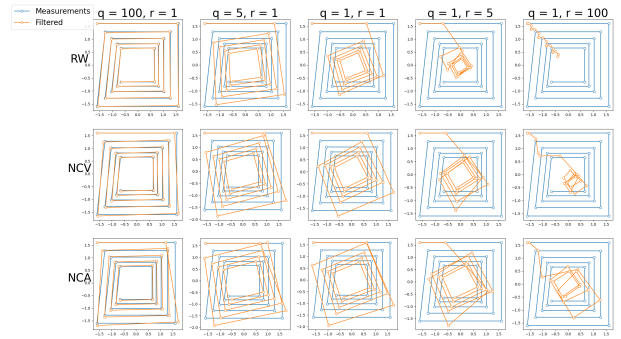


Figure 2. Kalman filtering method on the square spiral curve

was set to 0.8 of the size of the smaller dimension of the patch, and the number of particles was set to 100. Additionally, we used the Epanechnikov kernel on the patch before histogram extraction and employed the Hellinger distance to compute the similarity between histograms. Furthermore, we ensured that particle positions were clipped if they were assigned a location outside of the image.

Metric	Value
Average Overlap	0.52
Total Failures	32
Average Speed (FPS)	86.76

Table I
BASIC PARTICLE TRACKER PERFORMANCE

C. Testing using different motion models

We tested the tracker using all three of the aforementioned motion models. For each model, we experimented with different values for the dynamic noise constant q . Note that here q represents the ratio of the actual dynamics noise constant to the smaller dimension of the patch.

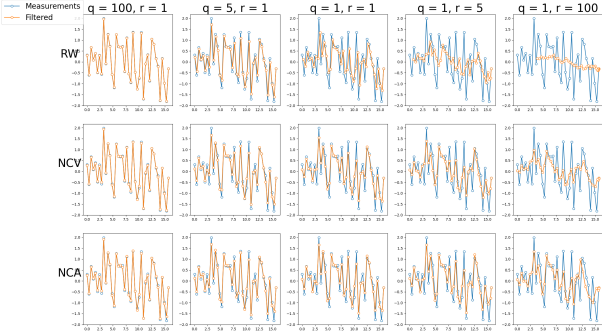


Figure 3. Kalman filtering method on random gaussian noise

The number of failures and average overlap for each motion model and different values of q are shown in Figure 4. It is evident that the RW and NCV models significantly outperform the NCA model, which struggles in both the number of failures and average overlap. The RW and NCV both manage to reach around 30 failures and average overlap at around 0.5.

We can see that for the RW and NCV models, a q value around 1 works best, while for the NCA model, a smaller q is preferable (Note that while the average overlap is larger at larger q , that is the consequence of the tracker being reset more times). This is because assigning a large uncertainty to the RW and NCV models helps the tracker rely more on the measurements, which is beneficial for these simpler motion models. In contrast, for the NCA model, a larger q increases the uncertainty in the acceleration, making the particle positions more erratic. This leads to many particles being pushed outside the target area, resulting in a higher number of failures. Therefore, a smaller q is more effective in the NCA model, as it reduces this unpredictability and keeps the particles within a more controlled range.

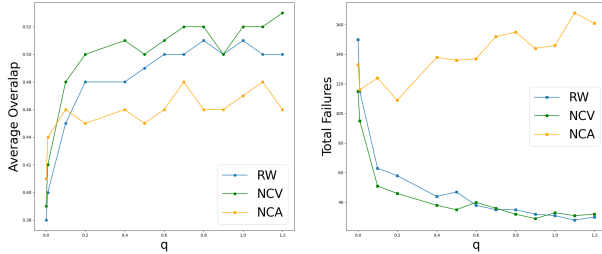


Figure 4. Tracker performance with different motion models and different dynamic noise constants

D. Testing the influence of number of particles

We also tested the dependence of performance on the number of particles (using the NCV motion model). In Figure 5, we show the average overlap, total failures, and average speed as functions of the number of particles. We observe that with a small number of particles, the tracker's performance drops significantly. However, after reaching approximately 30 to 50 particles, further increases in the number do not result in

significant improvements in performance. On the other hand, the average speed decreases at a rate proportional to 1×10^{-1} , as the computation time increases linearly with the number of particles.

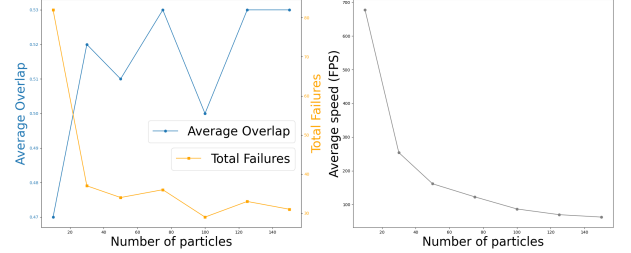


Figure 5. Tracker performance with different numbers of particles

E. Testing different color spaces

Next, we tested the performance of the tracker using different color spaces for creating the histograms (using the NCV motion model). The results are shown in Table II. Note that the parameters were optimized for the BGR color space, so the results may be slightly biased.

Color Space	Average Overlap	Total Failures
BGR	0.50	29.0
RGB	0.50	29.0
LAB	0.53	38.0
HSV	0.53	38.0
YCrCb	0.52	43.0

Table II

PERFORMANCE COMPARISON OF DIFFERENT COLOR SPACES USING THE PARTICLE FILTER TRACKER.

III. CONCLUSION

We found that when using the Kalman filter, the dynamic noise constant should be lower for the Random Walk (RW) motion model compared to the Nearly Constant Velocity (NCV) and Nearly Constant Acceleration (NCA) models. This is because the RW model lacks flexibility and should rely more on the measurements.

The particle filter tracker performed well on the dataset. We observed that the RW and NCV models significantly outperformed the NCA model. For the RW and NCV models, an optimal dynamic noise constant $q \approx 1$ was determined, while the NCA model performed better with $q \approx 0.2$.

We also found that increasing the number of particles improved the tracker's performance. However, after approximately 40 particles, the improvements stagnated, while the computation speed continued to decrease.

Furthermore, we determined that the BGR and RGB color spaces outperformed other color spaces in terms of tracker performance.

IV. APPENDIX

Here we present the matrices derived for the motion models.

A. Random walk

$$\mathbf{X} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad \phi = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} \Delta t q & 0 \\ 0 & \Delta t q \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

B. Nearly constant velocity

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \Phi = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} \frac{\Delta t^3 q}{3} & 0 & \frac{\Delta t^2 q}{2} & 0 \\ 0 & \frac{\Delta t^3 q}{3} & 0 & \frac{\Delta t^2 q}{2} \\ \frac{\Delta t^2 q}{2} & 0 & \Delta t q & 0 \\ 0 & \frac{\Delta t^2 q}{2} & 0 & \Delta t q \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

C. Nearly constant acceleration

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \ddot{x} \\ \ddot{y} \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \Phi = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} \\ 0 & 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{Q} = \begin{bmatrix} \frac{\Delta t^5 q}{20} & 0 & \frac{\Delta t^4 q}{8} & 0 & \frac{\Delta t^3 q}{6} & 0 \\ 0 & \frac{\Delta t^5 q}{20} & 0 & \frac{\Delta t^4 q}{8} & 0 & \frac{\Delta t^3 q}{6} \\ \frac{\Delta t^4 q}{8} & 0 & \frac{\Delta t^3 q}{3} & 0 & \frac{\Delta t^2 q}{2} & 0 \\ 0 & \frac{\Delta t^4 q}{8} & 0 & \frac{\Delta t^3 q}{3} & 0 & \frac{\Delta t^2 q}{2} \\ \frac{\Delta t^3 q}{6} & 0 & \frac{\Delta t^2 q}{2} & 0 & \Delta t q & 0 \\ 0 & \frac{\Delta t^3 q}{6} & 0 & \frac{\Delta t^2 q}{2} & 0 & \Delta t q \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$